Hafta 2: Araç Sürüş ve Telemetri Simülasyonu

(Modüler Programlama, Diziler, Kapsülleme (Encapsulation) ve Raporlama)

Açıklama

Geçen hafta elektromobilimizin motorunu çalıştırmadan önceki "Start-Up" güvenlik protokollerini başarıyla tamamladınız. Bu hafta ise aracımız piste çıkıyor!

Bu projede, birinci haftadaki kod mantığınızı temel alarak etkileşimli bir terminal simülasyonu oluşturacaksınız. Araç başlatıldıktan sonra sonsuz bir döngüye girerek kullanıcıdan sürüş komutları alacak ve aracın telemetri verilerini anlık olarak güncelleyecektir.

Ayrıca bu hafta aracımızın sürüş dinamiklerini daha gerçekçi hale getiriyoruz: İvmeler sabit olmayacak, rastgele değerlerde gerçekleşecek ve yaptığınız tüm işlemler dizilerde (arrays) saklanarak size sürüş istatistikleri olarak sunulacak. **Ek olarak, yazdığınız bu modüler sistemi resmi TEKNOFEST standartlarına uygun bir şekilde raporlamanız beklenmektedir.**

1. Versiyon Kontrolü ve GitHub PR Yönetimi (ÖNEMLİ)

Bu proje için **yeni** bir GitHub Repository'si oluşturun. Bu hafta kodlarınızı yerelde (terminalde) birleştirmek yerine, sektör standardı olan **Pull Request (PR)** yöntemini kullanarak GitHub üzerinden birleştireceksiniz.

İzlemeniz Gereken İş Akışı:

1. GitHub'da yeni deponuzu oluşturun ve yerelde `git init` ile başlatıp ilk commitinizi `main` branch'ine atın.
2. Geliştirmeye başlamadan önce terminalinizde yeni bir geliştirme dalı oluşturun ve o dala geçin: `git checkout -b dev`
3. **Tüm kodlama, raporlama ve commit işlemlerinizi** `dev` **branch'i üzerinde yapın.**
4. Kodlamanız ve raporunuz bittiğinde `dev` dalınızı GitHub'a gönderin: `git push origin dev`
5. GitHub'daki deponuza gidin. Yukarıda beliren **"Compare & pull request"** butonuna tıklayarak `dev` dalındaki kodlarınızı `main` dalına aktarmak için bir Pull Request açın.
6. Kodunuzu inceleyin, hatasız olduğundan eminseniz **"Merge pull request"** butonuna basarak kodlarınızı ana dalda birleştirin.

2. Modüler Dosya Mimarisi ve Durum Yönetimi (Kapsülleme)

Projenizde aşağıdaki 3 dosya bulunmalıdır:

- `telemetry.h` **:** Fonksiyon prototiplerini ve sabitleri (örn: `#define MAX_KAYIT 100`) barındıracak.

- `telemetry.c` **:** Aracın anlık durumu (hız, batarya, motor sıcaklığı, batarya sıcaklığı), işlem geçmişini tutan diziler ve bu verileri güncelleyen fonksiyonların asıl gövdeleri burada yer alacak.
- `main.c` **:** Start-Up kontrolünü (1. Hafta kodları) ve sadece ana menü döngüsünü (switch-case) barındıracak. Değişkenler burada tutulmayacak, menü üzerinden sadece ilgili fonksiyonlar çağrılacak.

C dilinde fonksiyonlara parametre gönderdiğimizde, değişkenlerin sadece "kopyaları" gider. Orijinal verileri değiştirebilmek için bu hafta **"Module-Level State" (Modül Seviyesi Durum/Kapsülleme)** mantığını kullanacağız.

Aracınızın anlık verilerini (hız, batarya, sıcaklıklar) ve geçmişi tutan dizileri `main.c` içinde **DEĞİL**, `telemetry.c` dosyasının en üst kısmında tanımlamalısınız. Böylece telemetri fonksiyonları bu verilere doğrudan erişip onları değiştirebilir.

3. Gömülü Sistem Standartları: Neden Global Değişken ve Pointer Yasak?

Bu ödevde, fonksiyonların dışında (`main.c` veya diğer dosyalarda her yerden erişilebilen) **Global Değişken kullanmak kesinlikle YASAKTIR**. Ayrıca bu hafta **Pointer (`*` , `&`) kullanımı da kısıtlanmıştır.** Bu kuralların ardında otomotiv yazılımı (MISRA C) ve gömülü sistem mühendisliğinin temel güvenlik prensipleri yatar:

- **Global Değişken Tehlikesi (Side Effects):** Bir elektromobil gibi "Safety-Critical" (güvenlik açısından kritik) sistemlerde global değişkenler ölümcül riskler taşır. Hangi fonksiyonun, ne zaman ve nereden değişkeni değiştirdiğini (örneğin `hiz` değişkenini) takip etmek imkansızlaşır. Bu durum "spagetti koda" ve tespiti çok zor bug'lara yol açar.
- **Kapsülleme (Encapsulation) Zorunluluğu:** Veriyi sadece onunla ilgilenen modüle (`telemetry.c`) hapsetmek (kapsüllemek), o verinin izinsiz veya kazara değiştirilmesini engeller. Bu, güvenilir kod yazmanın ilk adımıdır.
- **Pointer Hataları (Hard Fault):** Pointer'lar gömülü sistemlerin kalbidir ancak bellek yönetimi (Memory Management) konusu tam kavranmadan kullanıldıklarında; boş bir adrese yazma, "Dangling Pointer" veya bellek sızıntısı (Memory Leak) gibi durumlara sebep olurlar. Bir bilgisayarda bu durum "Segmentation Fault" verip programı kapatır, ancak STM32 gibi bir mikrodenetleyicide aracı hareket halindeyken kilitleyerek (Hard Fault) kazaya sebep olur. Bu konulara önümüzdeki haftalarda güvenli bir şekilde giriş yapılacaktır.

4. İpucu: Örnek Fonksiyon Taslakları

Araç verileri `telemetry.c` içinde tutulacağı için fonksiyonların parametre almasına gerek yoktur. Sadece işlemleri tetiklemeleri yeterlidir. `telemetry.h` dosyasında bulunabilecek örnek bir yapı aşağıdadır:

```
// telemetry.h dosyası örnek içeriği
#ifndef TELEMETRY_H
#define TELEMETRY_H

#define MAX_KAYIT 100
```

```
// Fonksiyon Prototipleri (İmzaları)
void sistemi_kur(float baslangic_sicaklik, int baslangic_sarj);
void gaza_bas();
void frene_bas();
void rejen_fren();
void telemetri_ve_istatistik_yazdir();
void sistemi_kapat();

#endif
```

5. Görevler ve İş Mantığı

Aşama 1: Başlangıç, Sıcaklıklar ve Rastgelelik

- Programınız önce Hafta 1'deki sensör veri alma ve kontrol adımlarını yapmalıdır. Araç "SİSTEM HAZIR" mesajı verdikten sonra simülasyona geçilmelidir.
- Simülasyon başlangıç hızı `0.0` (float) olmalıdır.
- **ÖNEMLİ:** Hafta 1'de kullanıcıdan aldığınız `batarya_sicakligi` değerini hem bataryanın hem de motorun başlangıç (ortam) sıcaklığı olarak kabul edin. Kullanıcıdan alınan bu başlangıç değerlerini `sistemi_kur(sicaklik, sarj)` fonksiyonuna göndererek `telemetry.c` içindeki modül değişkenlerine atayın.
- Rastgele sayı üretebilmek için `main.c` başında `srand(time(NULL));` fonksiyonunu çağırmayı unutmayın (`<stdlib.h>` ve `<time.h>` gereklidir).

Aşama 2: Dizilerle Veri Kaydı `telemetry.c` içinde her bir eylemin geçmişini tutmak için `float` tipinde 3 ayrı dizi ve bu dizilerin kaç elemanla dolduğunu takip eden sayaç değişkenleri tanımlayın:

- `hizlanma_kayitlari[MAX_KAYIT]`
- `yavaslama_kayitlari[MAX_KAYIT]`
- `rejen_kayitlari[MAX_KAYIT]`

Aşama 3: Menü Seçenekleri ve Termal Etkileri

1. **Gaza Bas:** Aracı 2.0 ile 10.0 km/s arasında (0.1 adımlarla) rastgele bir değer kadar hızlandırır ve bu değeri diziye (array) ekler.
   - Yüksek akım çekildiği için Bataryayı %2 azaltır.
   - **Termal Etki:** Motor sıcaklığını 5.0°C artırır, Batarya sıcaklığını 2.0°C artırır.
2. **Frene Bas:** Hızı 5.0 ile 15.0 km/s arasında (0.1 adımlarla) rastgele bir değer kadar düşürür ve diziye ekler.
   - **Termal Etki:** Motora yük binmediği ve rüzgar soğutması olduğu için Motor sıcaklığı 3.0°C azalır, Batarya sıcaklığı 1.0°C azalır.
3. **Rejeneratif Frenleme:** Hızı 2.0 ile 10.0 km/s arasında (0.1 adımlarla) rastgele bir değer kadar düşürür ve diziye ekler.
   - Kinetik enerjiyi geri kazanarak Bataryayı %1 artırır.

- **Termal Etki:** Motor jeneratör modunda çalıştığı için normal frene göre daha az soğur (Motor sıcaklığı 2.0°C azalır). Batarya ise şarj edildiği için ısınır (Batarya sıcaklığı 1.0°C artar).

4. **Telemetriyi ve İstatistikleri Oku:** O anki verileri ve geçmiş istatistikleri ekrana yazdırır.
5. **Sistemi Kapat:** Çıkış yapmadan önce son bir kez genel sürüş istatistiklerini ekrana yazdırır ve ardından döngüyü kırarak programı güvenli şekilde sonlandırır.

(İpucu: 2.0 ile 10.0 arasında 0.1 adımlı rastgele bir float üretmek için tam sayıları 10'a bölebilirsiniz: `(rand() % 81 + 20) / 10.0f`)

6. İstatistik Hesaplama (Telemetri Ekranı)

Kullanıcı 4. seçeneği seçtiğinde dizilerdeki veriler kullanılarak hesaplanmış şu istatistikler basılmalıdır:

- O anki güncel Hız (örn: `45.3 km/s`), Batarya Yüzdesi, **Motor Sıcaklığı** ve **Batarya Sıcaklığı**.
- Toplam gaza basma sayısı ve ortalama hızlanma ivmesi (Dizideki elemanların ortalaması).
- Toplam frene basma sayısı ve ortalama yavaşlama ivmesi.
- Toplam rejeneratif frenleme sayısı ve ortalama yavaşlama miktarı (Dizideki elemanların ortalaması).

7. Sınır Durumlar (Edge Cases) ve Hata Yönetimi

Simülasyonunuz fiziksel kurallara ve kullanıcı hatalarına karşı dayanıklı olmalıdır:

- **Maksimum Hız:** Aracımızın ulaşabileceği maksimum hız **70.0 km/s**'dir. Hızlanma eylemi bu değeri aşarsa hız 70.0'a sabitlenmelidir.
- **Negatif Hız Olamaz:** Araç duruyorken (0.0 km/s) frene basılırsa uyarı verin. Hareket halindeyken yavaşlama miktarı mevcut hızdan büyükse, hızı eksiye düşürmek yerine doğrudan `0.0` 'a eşitleyin.
- **Batarya Sınırları (Taşma ve Düşme):** * Rejeneratif frenleme yaparken batarya %100'ü aşmamalıdır.
  - Batarya %1 iken gaza basılırsa batarya eksiye düşmemeli, tam `0` 'da kalmalı ve "Batarya Tükendi" uyarısı verilerek araç hareket ettirilmemelidir.
- **Termal Sınırlar (Aşırı Isınma ve Aşırı Soğuma):**
  - **Kritik Isınma:** Motor sıcaklığı 90.0°C'yi veya Batarya sıcaklığı 70.0°C'yi aşarsa sistem "KRİTİK HATA: Aşırı Isınma!" verip programı acil kapatmalıdır.
  - **Soğuma Sınırı:** Araç sürekli fren yapsa bile motor ve batarya sıcaklıkları ortam sıcaklığının (Bu simülasyonda 20.0°C olarak kabul edin) altına düşmemelidir.
- **Terminalin Çökmesi (Kritik Input Hatası):** Menüde `int` bir sayı (1-5) beklenirken kullanıcı harf (örn: 'A') girerse C dilindeki `scanf` fonksiyonu sonsuz döngüye girer. Bu bug'ı çözmek (buffer'ı temizlemek) sizin sorumluluğunuzdadır!

- **Dizi Sınırları:** Kullanıcı 100 kereden fazla bir işlem yaparsa diziler dolacağı için program çökmemeli, 100'den sonra o eylem için yeni istatistik kaydetmeyi durdurmalıdır.

8. Kod Açıklama Raporu

Bu hafta yazdığınız kodların çalışır durumda olmasının yanında, kurduğunuz modüler yapıyı ve algoritmaları anlatan bir rapor yazmanız zorunludur. Rapor dosyanızı (`Rapor.pdf` veya `Rapor.docx` formatında) repository'nize eklemelisiniz.

ÖNEMLİ: Raporunuz, sizinle ekte paylaşılan **"9-Efficiency_Challenge_Elektrikli_Araç_Yarışları_2025_Kurallar_Bölüm_YdooA.pdf" (Bölüm 9: Rapor Yazım Kılavuzu)** şartnamesine **birebir** uygun olmalıdır.

Rapor Yazım Kılavuzu'ndan Dikkat Etmeniz Gereken Bazı Temel Şartlar:

- **Biçim:** Yazı tipi *Arial*, boyutu *12 punto*, satır aralığı *1.15* olmalıdır. Kenar boşlukları her yönden *2.5 cm* bırakılmalı ve metin iki yana yaslanmalıdır.
- **İçerik:** Rapor; yazdığınız `telemetry.c` dosyasındaki fonksiyonların amacını, dizileri nasıl kullandığınızı ve sınır durumlarını (Edge Cases) kodda nasıl çözdüğünüzü açıklamalıdır.
- **Etik Kurallar:** Rapor içeriğinde yapay zeka (AI/GPT) kullanımı **yasaktır**. Raporun benzerlik oranı alıntılar hariç **%20'yi geçmemelidir**. Toplamda 1 sayfayı aşan "kopyala-yapıştır" metin tespiti durumunda **tarafınıza ciddi bir uyarı yapılacak ve raporun kurallara uygun şekilde baştan revize edilmesi istenecektir.**

Örnek Çıktı

```
--- ELEKTROMOBİL GÜVENLİK KONTROLÜ ---
Batarya Sıcaklığı (°C): 35.0
Şarj Yüzdesi (%): 85
Kapı Durumu (A/K): K
Fren Pedalı (1:Basılı, 0:Değil): 1

SONUÇ: BAŞARILI: SİSTEM HAZIR. MOTOR BAŞLATILIYOR...

--- TUFAN ELEKTROMOBİL SÜRÜŞ SİMÜLASYONU ---
1. Gaza Bas
2. Frene Bas
3. Rejeneratif Frenleme Yap
4. Anlık Telemetri ve İstatistikleri Oku
5. Sistemi Kapat
Seçiminiz: 1
[BİLGİ] Gaza basıldı. Araç 5.4 km/s hızlandı. Motor: 40.0°C, Batarya: 37.0°C

--- TUFAN ELEKTROMOBİL SÜRÜŞ SİMÜLASYONU ---
1. Gaza Bas
2. Frene Bas
...
Seçiminiz: 3
[BİLGİ] Rejeneratif fren devrede. Araç 4.1 km/s yavaşladı. Batarya şarj oluyor.

...
Seçiminiz: 4
```

```
--- ANLIK TELEMETRI ---
Güncel Hız: 21.3 km/s
Batarya: %82
Motor Sıcaklığı: 43.0 °C
Batarya Sıcaklığı: 39.0 °C
-----------------------
--- SÜRÜŞ İSTATİSTİKLERİ ---
Gaza Basma Sayısı: 3 (Ortalama Artış: 6.75 km/s)
Frene Basma Sayısı: 1 (Ortalama Düşüş: 6.20 km/s)
Rejeneratif Frenleme Sayısı: 1 (Ortalama Düşüş: 4.10 km/s)
-----------------------

...
Seçiminiz: 5
[BİLGİ] Sistem kapatılıyor... Son Sürüş İstatistikleri:
--- SÜRÜŞ İSTATİSTİKLERİ ---
Gaza Basma Sayısı: 3 (Ortalama Artış: 6.75 km/s)
Frene Basma Sayısı: 1 (Ortalama Düşüş: 6.20 km/s)
Rejeneratif Frenleme Sayısı: 1 (Ortalama Düşüş: 4.10 km/s)
-----------------------
[BİLGİ] Motor güvenli bir şekilde kapatıldı. İyi günler!
```